Estrutura de Dados

Aula 05 – Listas Encadeadas: Estruturas Dinâmicas

ibmec

Quem sou eu?

Professor: Luís Aramis dos Reis Pinheiro.

- **Doutorado e Mestrado** em **Ciências Mecânicas** – **UnB** – **CAPES 7**
- **Graduação** em Licenciatura em Física – **UNIFAP**

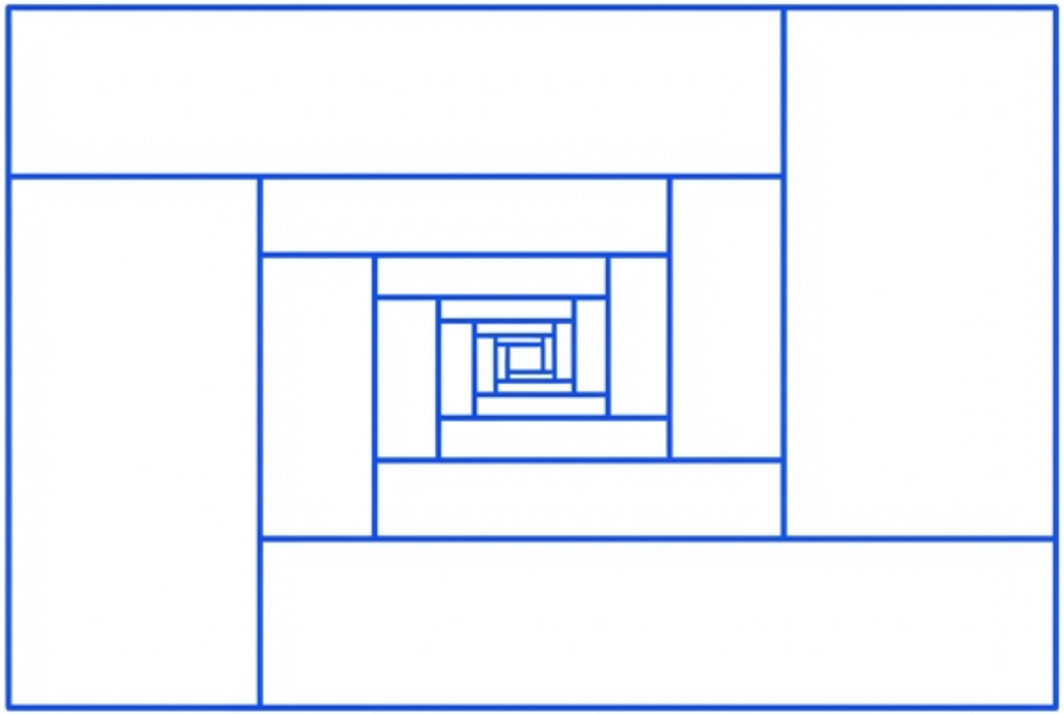
luis.pinheiro@professores.ibemec.edu.br

(96) 99907-5819

@l_aramis

Recursividade e Análise de Algoritmos

Abordagem Iterativa vs. Recursiva



Disciplina: Estruturas de Dados (IBM3114)
Aula 03 | Tempo 01

Round 1: A Abordagem Iterativa

```python
def fatorial_iterativo(n):
    resultado = 1
    for i in range(1, n + 1):
        resultado *= i
    return resultado
```

- **Estratégia:** Uso de um loop explícito (`for` ou `while`).
- **Memória:** $O(1)$ (Constante).
- **Características:** Direto, eficiente e baixo consumo de memória.

Implementação em Python

```python
def fatorial_recursivo(n):
    if n < 2:
        return 1
    else:
        return n * fatorial_recursivo(n - 1)
```

Caso Base

Passo Recursivo

Característica	Recursividade	Loop (Iteração)
Definição	Uma função que chama a si mesma para resolver partes menores do problema.	Um bloco de código que se repete continuamente até que uma condição seja falsa.
Condição de Parada	Termina quando atinge um **caso base** (condição de saída explícita).	Termina quando a **condição do loop** falha.
Uso de Memória	**Alto.** Cada chamada recursiva adiciona uma nova camada à pilha de execução (*call stack*).	**Baixo.** Utiliza a mesma região de memória durante toda a execução.
Desempenho / Velocidade	Geralmente **mais lento** devido ao tempo extra (*overhead*) de criar e destruir as chamadas na pilha.	Geralmente **mais rápido**, pois não há o *overhead* de múltiplas chamadas de função.
Risco Principal	**Stack Overflow** (estouro de pilha de memória) se o caso base falhar ou demorar muito.	**Loop Infinito** (trava a execução) se a condição de parada nunca for alcançada.
Tamanho / Legibilidade	Código costuma ser mais **curto, limpo e elegante**, especialmente em problemas matemáticos complexos.	Código pode ser mais **longo**, mas a lógica passo a passo é geralmente mais fácil de acompanhar para iniciantes.
Casos de Uso Ideais	Estruturas de dados complexas (Árvores, Grafos), algoritmos de Dividir para Conquistar (Merge Sort, Quick Sort).	Tarefas repetitivas simples, iteração em listas/arrays, contadores.

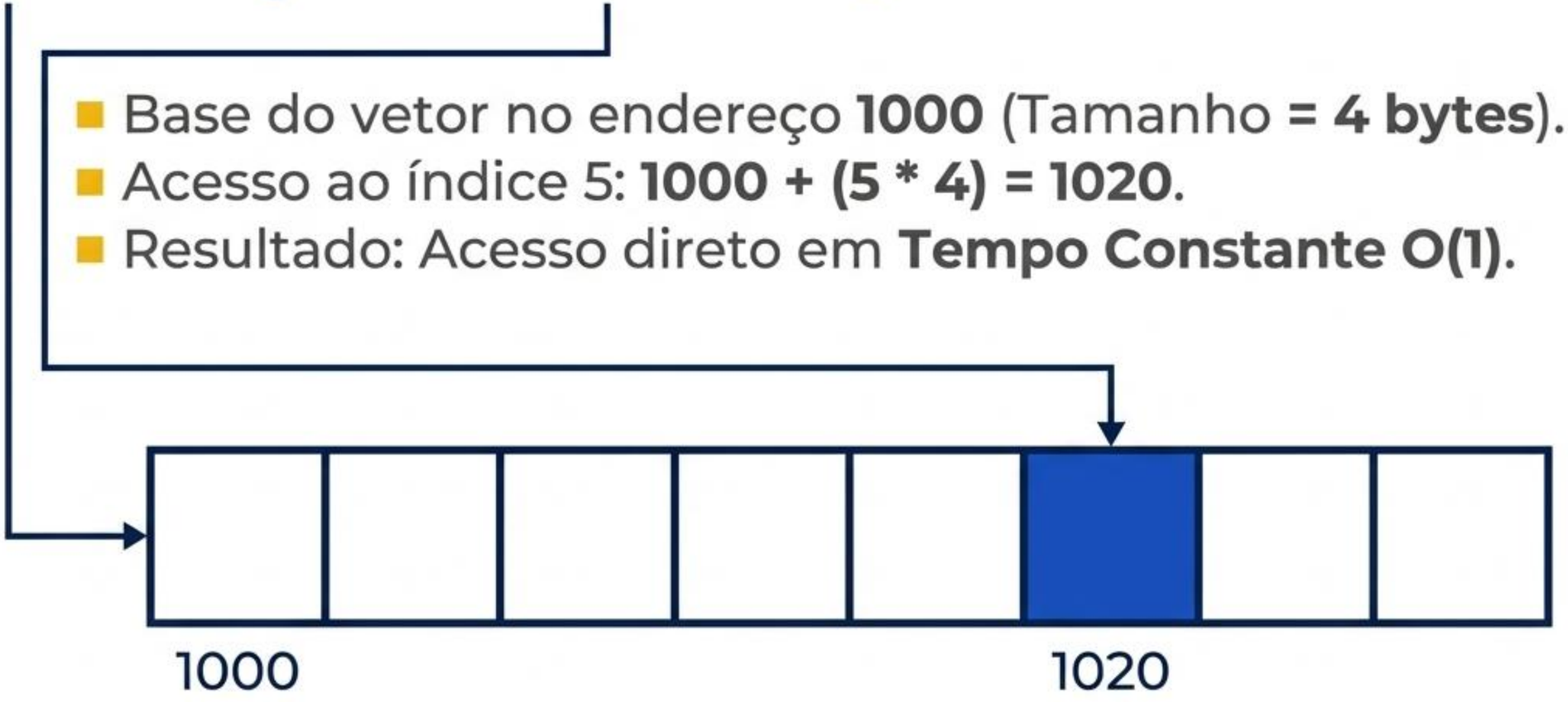
Vetores e Memória Física

Como o hardware enxerga as suas variáveis.

A Matemática do O(1) na Memória

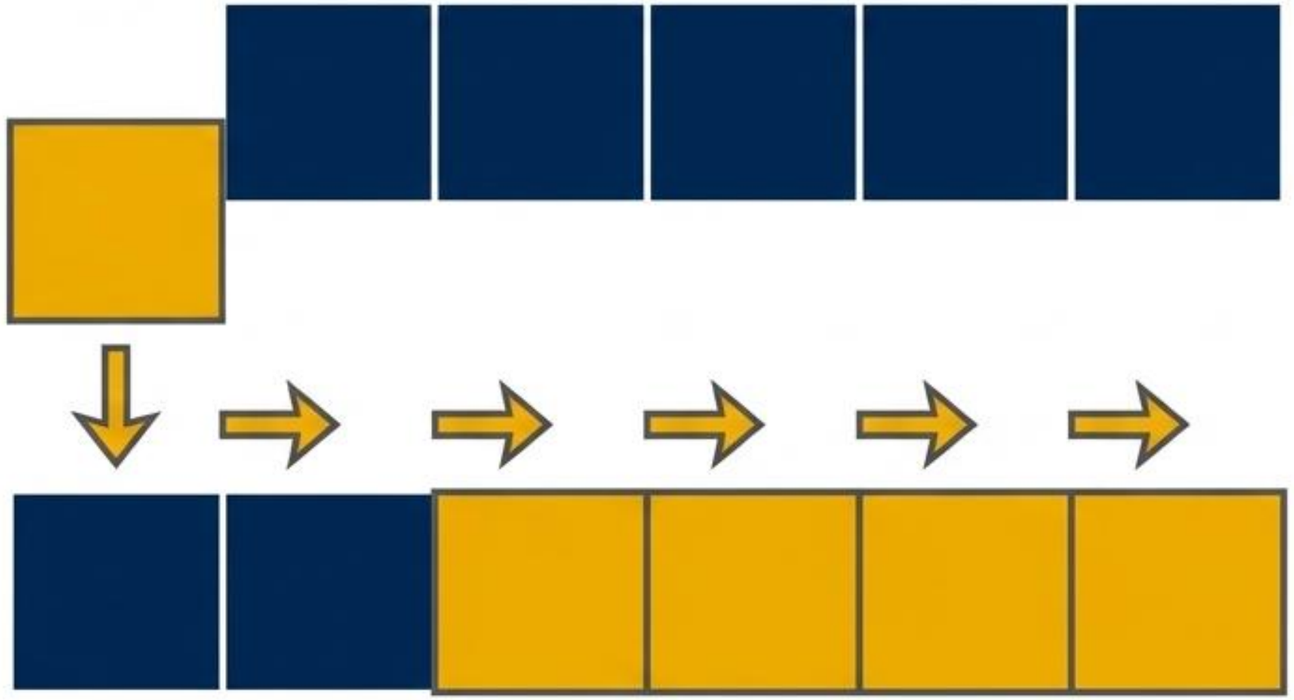
Endereço = Base + (Índice * Tamanho)

- Base do vetor no endereço **1000** (Tamanho **= 4 bytes**).
- Acesso ao índice 5: **1000 + (5 * 4) = 1020**.
- Resultado: Acesso direto em **Tempo Constante O(1)**.



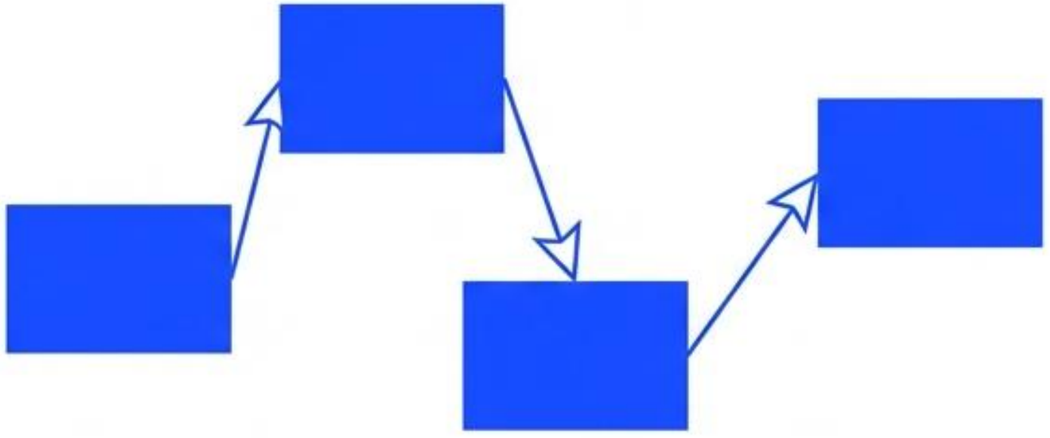
O Preço da Rigidez (Custo do Deslocamento)

- A memória contígua não permite expansão interna.
- Inserir um novo elemento exige espaço.
- O efeito cascata: Deslocamento obrigatório para a direita.



Listas Encadeadas: Estruturas Dinâmicas

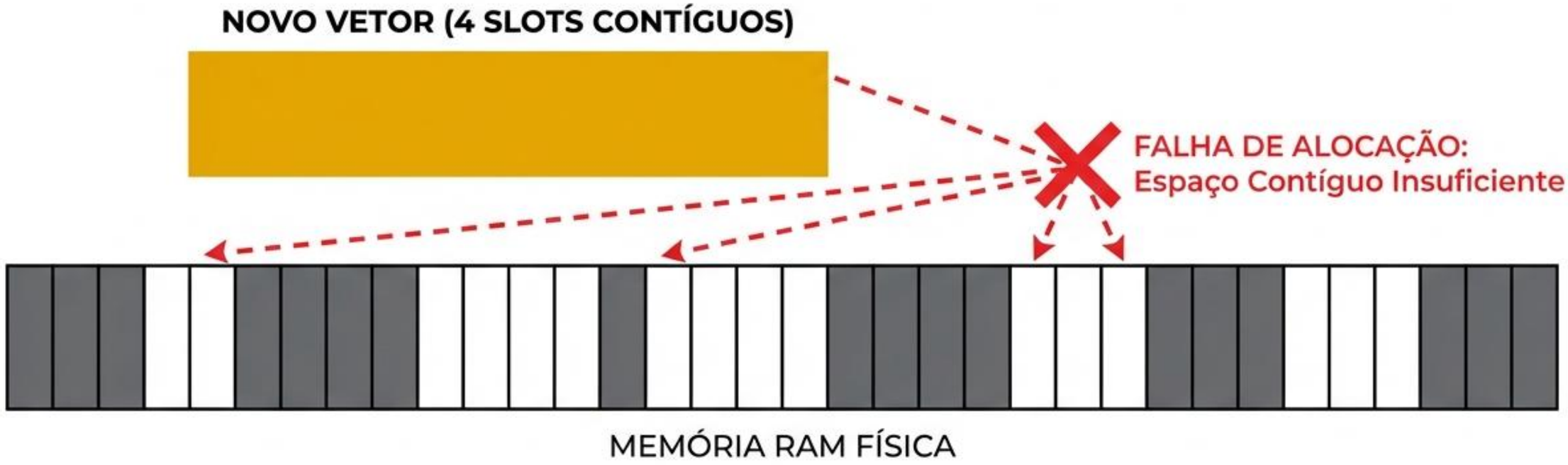
- O fim da memória contígua.
- Transição de índices matemáticos para mapeamento dinâmico.
- A lógica da "caça ao tesouro" na memória RAM.



Linked Lists

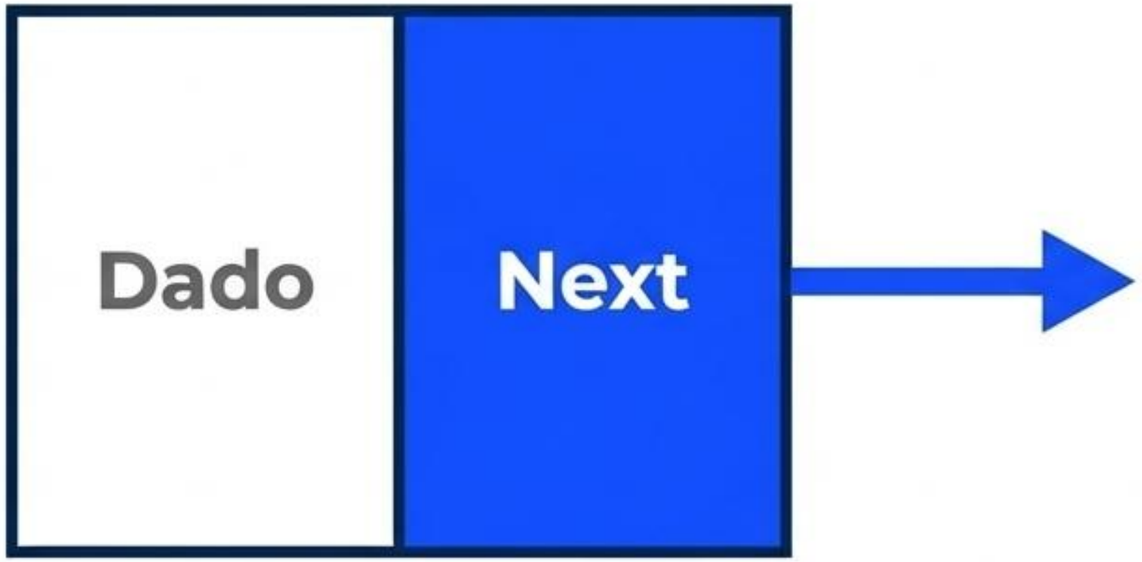
O Problema do Vetor e a Fragmentação

- Vetores exigem blocos contíguos gigantes na memória.
- Memória RAM fragmentada impede novas alocações estáticas.
- O desperdício de espaço no redimensionamento.



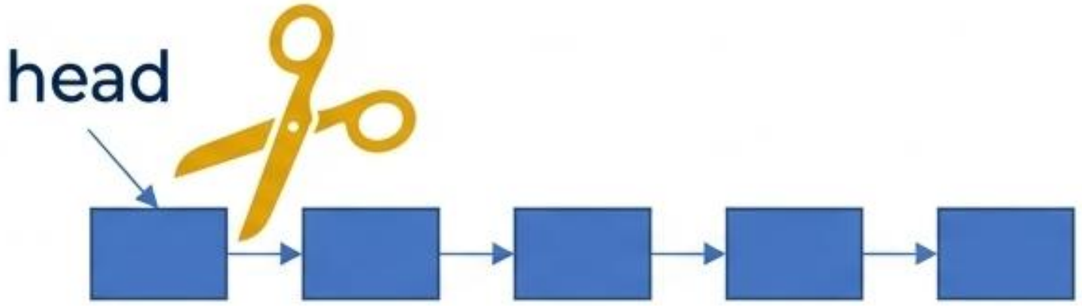
A Solução: Anatomia do Nó (Node)

- Dados espalhados dinamicamente sob demanda.
- **Node (Nó)**: A unidade fundamental da lista.
- **Valor (Data)**: Onde a informação real fica armazenada.
- **Ponteiro (Next)**: O endereço de memória do próximo elemento.



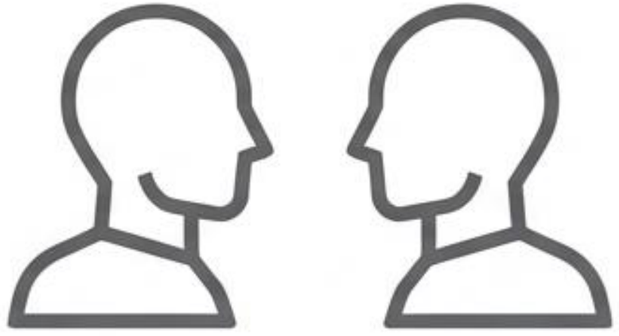
Teste Conceitual: O Elo Perdido

- Acesse o Socrative (Votação Individual e Cega).
- Cenário: Temos uma Lista Encadeada com 5 nós interligados.
- Pergunta: O que acontece com a memória se a variável head (a cabeça da lista) for apagada ou sobrescrita acidentalmente?



Conflito de **Respostas:** Debate em Pares

03:00



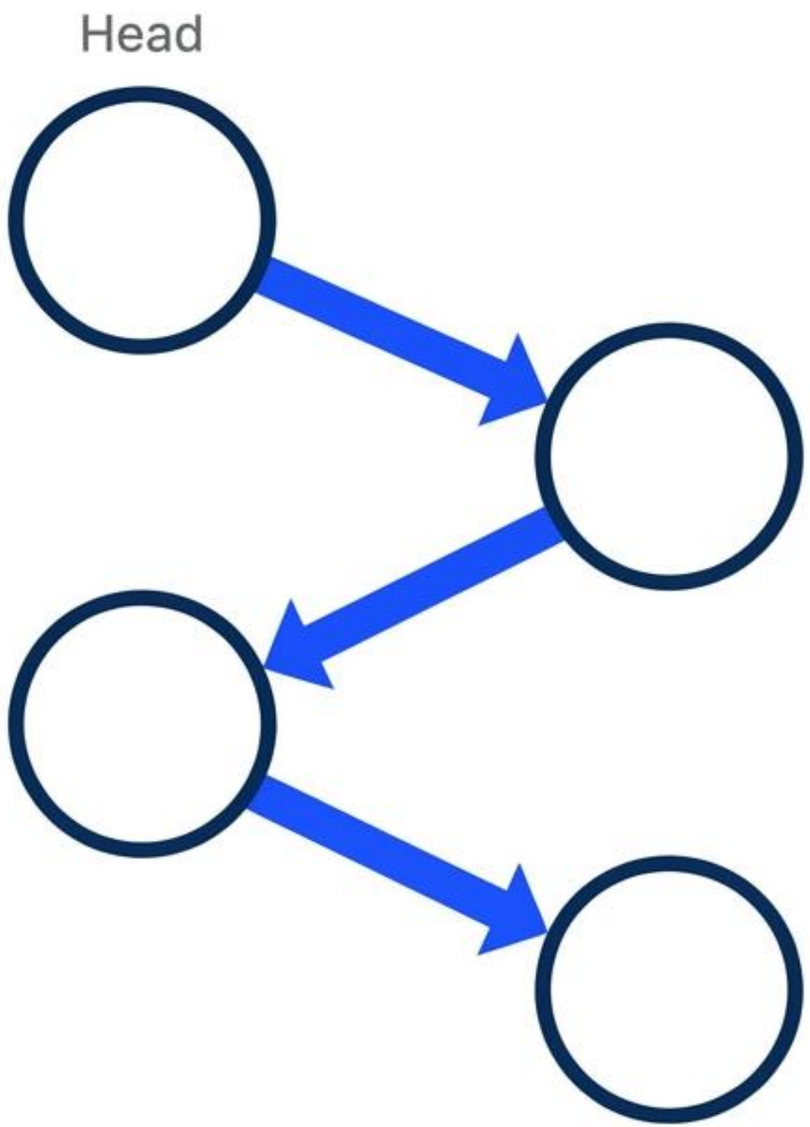
- Junte-se ao colega do lado.
- Defenda sua lógica utilizando os conceitos de **Ponteiros e Acesso Sequencial.**
- Preparem-se para a **Revotação.**

Resolução: O Risco do Memory Leak

- Todos os dados são perdidos.
- O acesso em listas é estritamente sequencial (não há índice).
- **Memory Leak (Vazamento de Memória)**: Os nós continuam ocupando a RAM, mas ficam **inatingíveis**.
- A importância do controle rigoroso de referências.

Costurando os Dados na Memória

- A quebra do paradigma da memória contígua.
- A Lista Encadeada como solução de alocação dinâmica.
- A nova guerra de complexidades computacionais.
- Trade-offs de processamento: O(1) vs O(n).

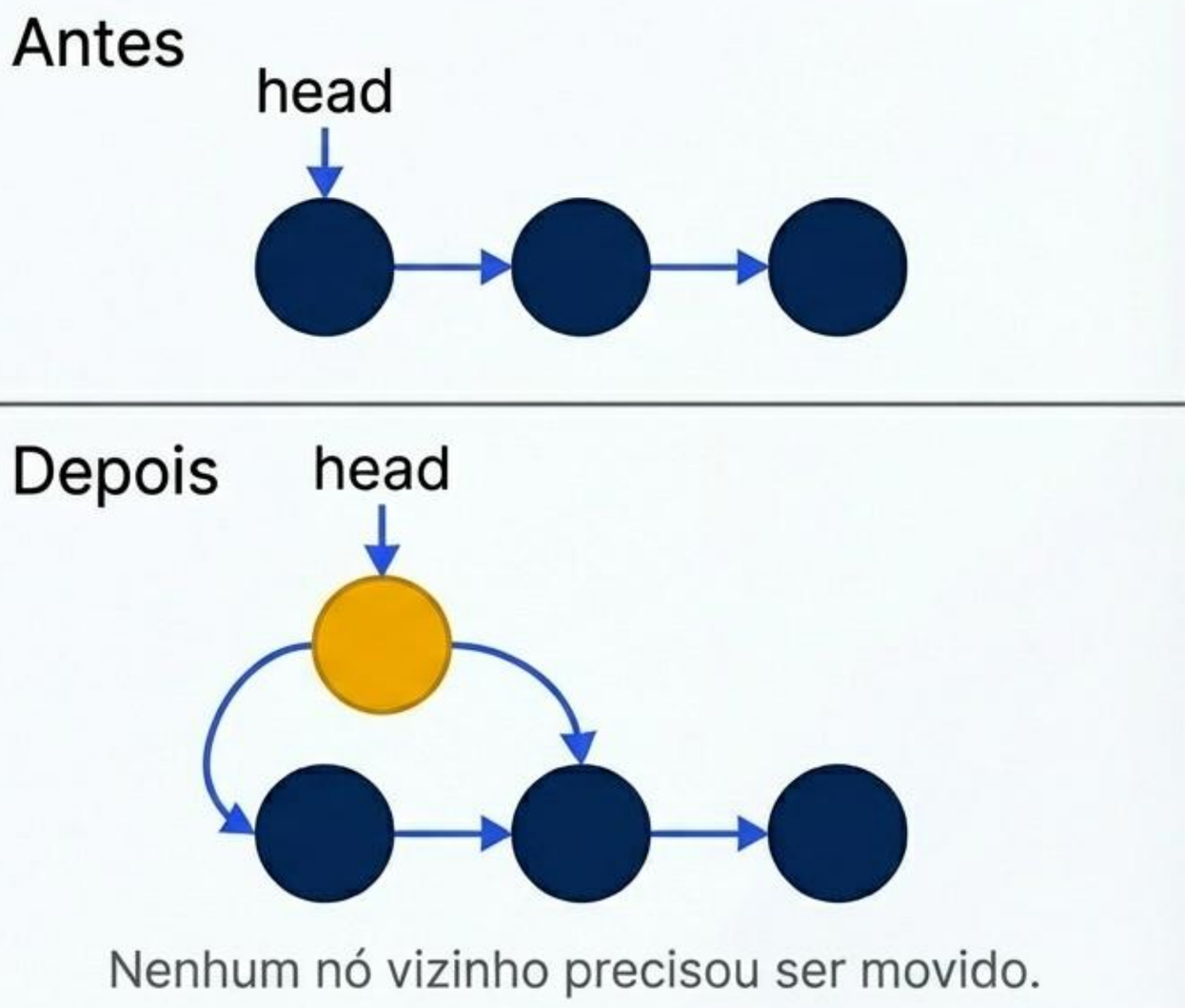


Painel de Batalha: Vetores vs. Listas

	Vetores	Listas Encadeadas
Acesso e Busca	Matemático e Imediato **O(1)**	Travessia Sequencial O(n)
Inserção no Início	Deslocamento Físico O(n)	Ancoragem Direta **O(1)**
Gerenciamento Físico	Exige bloco único (Risco de falha)	Usa fragmentos soltos (Alta flexibilidade)

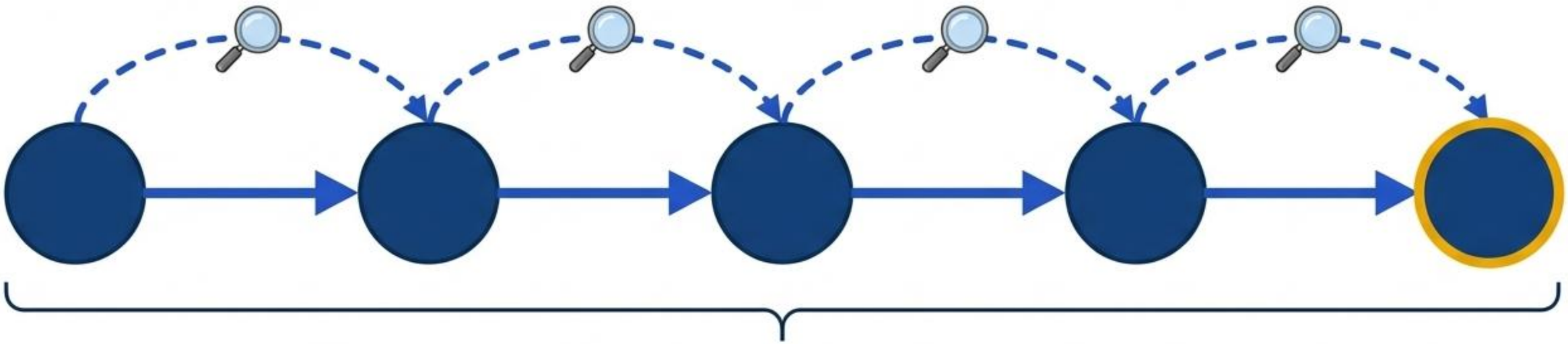
O Poder da Inserção em O(1)

- Custo temporal puramente constante.
- Ausência de deslocamento de memória física.
- Instanciação do nó e ancoragem imediata.



O Preço da Busca em O(n)

- Acesso estritamente sequencial aos dados.
- Impossibilidade técnica de busca indexada.
- O gargalo da leitura de ponteiros sucessivos.
- Custo linear no pior caso.

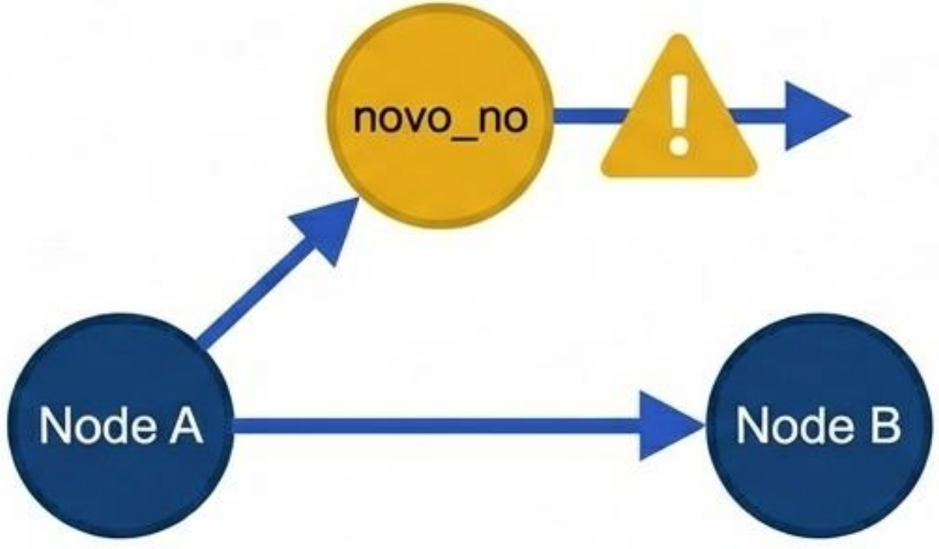


Acesso ao 5º elemento exige 4 saltos físicos na memória.

O Paradoxo da Inserção Central

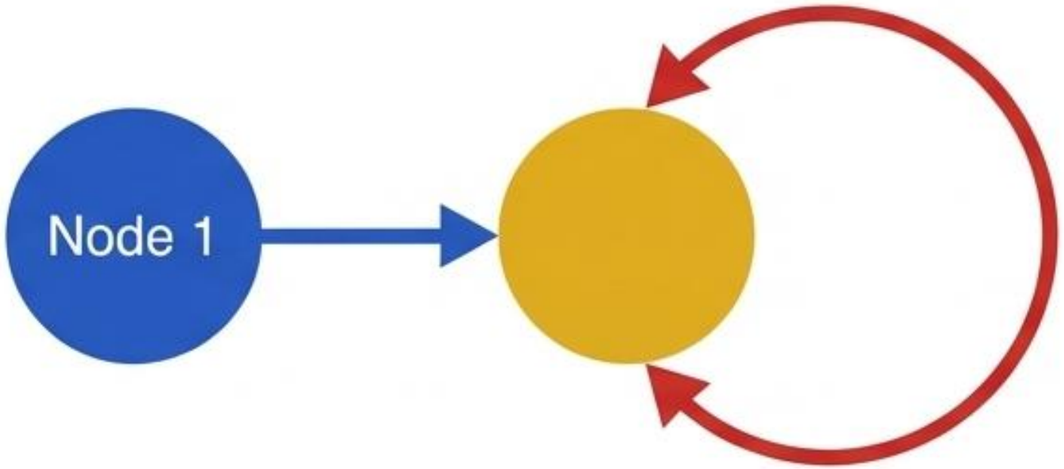
- Tentativa de inserção no meio da lista.
- Sobrescrita intuitiva da referência do nó anterior.
- Teste Socrative: O que acontece com a memória?

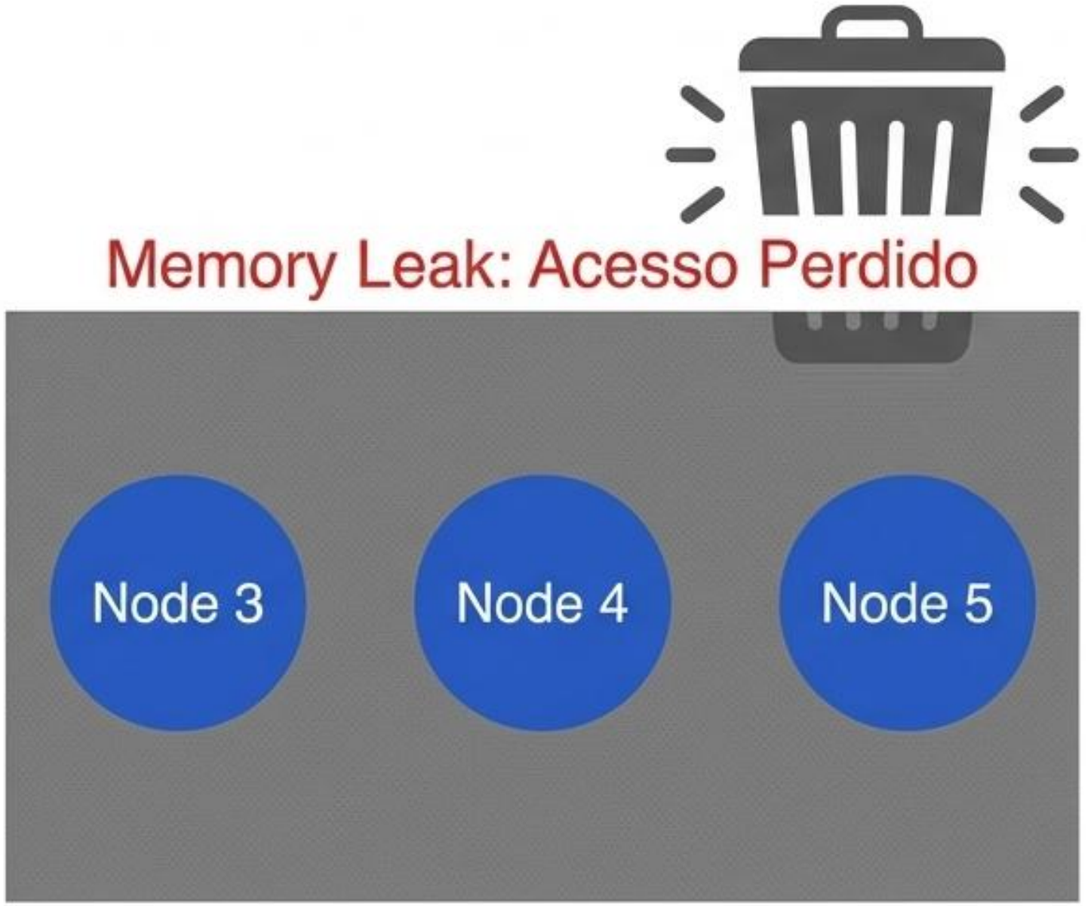
```
no_anterior.next = novo_no
novo_no.next = no_anterior.next
```



O Desastre do Vazamento de Memória

- Perda irrecuperável de referência.
- O abismo da memória isolada.
- Intervenção letal do Garbage Collector.





A Regra de Ouro das Listas Encadeadas

Nunca solte a corda antiga antes de segurar a nova.

Passo 1

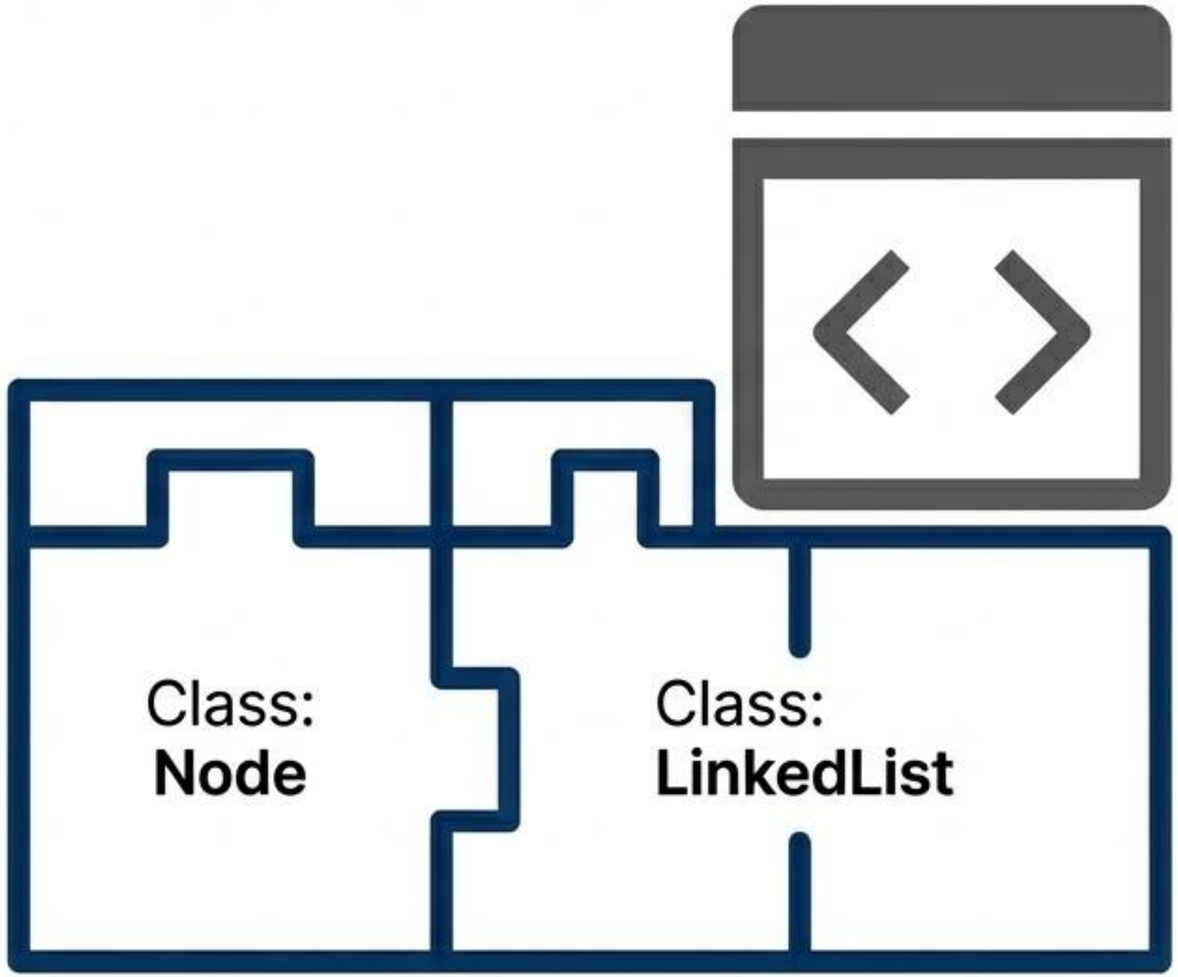
```
novo_no.next = no_anterior.next
```

Passo 2

```
no_anterior.next = novo_no
```

O Desafio do Laboratório

- Fundamentos da costura de dados dominados.
- Prevenção de rompimento assimilada.
- Modelagem Orientada a Objetos (Classes Node e LinkedList).
- Pair Programming: Implementação do zero.



Laboratório Prático: O Fio da Meada

Construindo Listas Encadeadas com Orientação a Objetos

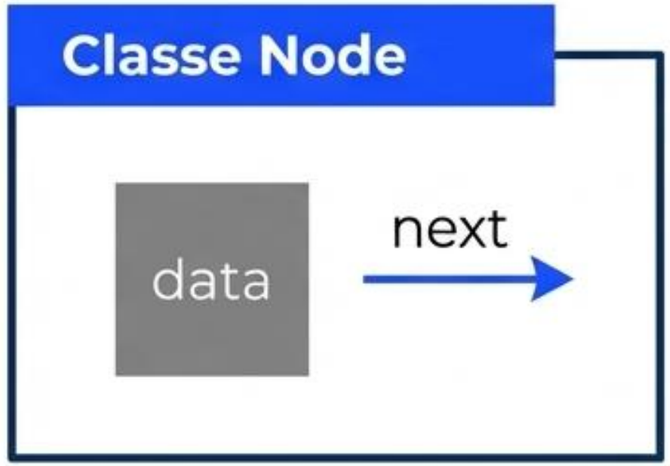
Alocação Dinâmica de Memória na Prática

Transição de vetores contíguos para nós alocados dinamicamente.

Implementação de operações fundamentais via Orientação a Objetos.

Validação da lógica de ponteiros na memória RAM.

Arquitetura Orientada a Objetos



- Atua como o contêiner de dados.
- Armazena o valor (data) e o ponteiro de conexão (next).

- Atua como a estrutura gestora.
- Mantém o rastreamento inicial através do ponteiro head.

Matriz de Responsabilidades: Pair Programming

O Piloto (Driver)

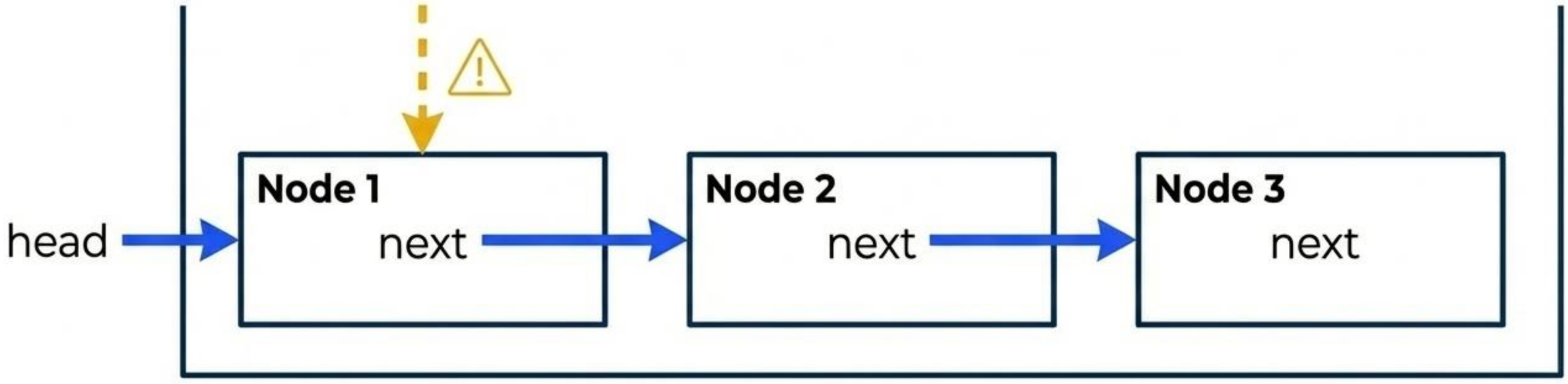
- Mãos exclusivas no teclado e mouse.
- Foco micro: Sintaxe limpa (PEP 8), digitação e execução.
- Transcreve a lógica para a linguagem Python.

O Copiloto (Navigator)

- Mãos fora do teclado.
- Foco macro: Arquitetura, fluxo de dados e revisão em tempo real.
- Dita a lógica estrutural e previne vazamentos de memória.

O professor não digita código, apenas resolve impasses lógicos.

O Blueprint da Memória Dinâmica



- head nunca contém dados, apenas o endereço do primeiro nó.
- A iteração (while current_node.next:) é o único motor de busca.
- **Regra Crítica: Nunca sobrescreva next antes de ancorar o novo nó.**

Início do Sprint de Codificação

O RELÓGIO ESTÁ RODANDO

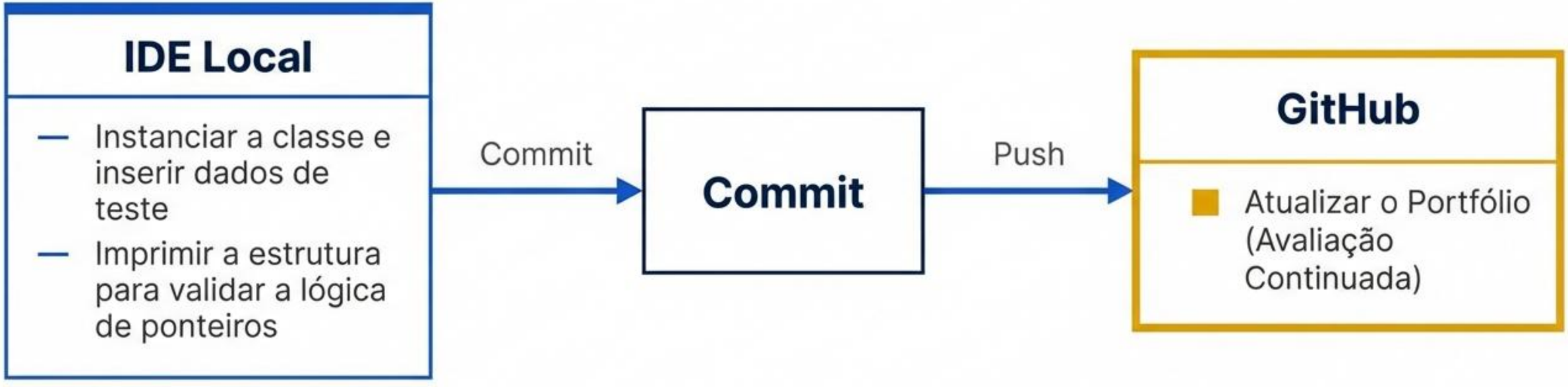
20:00

Abra o VS Code. Navegadores, iniciem a arquitetura da class Node.

Evolução e Fechamento

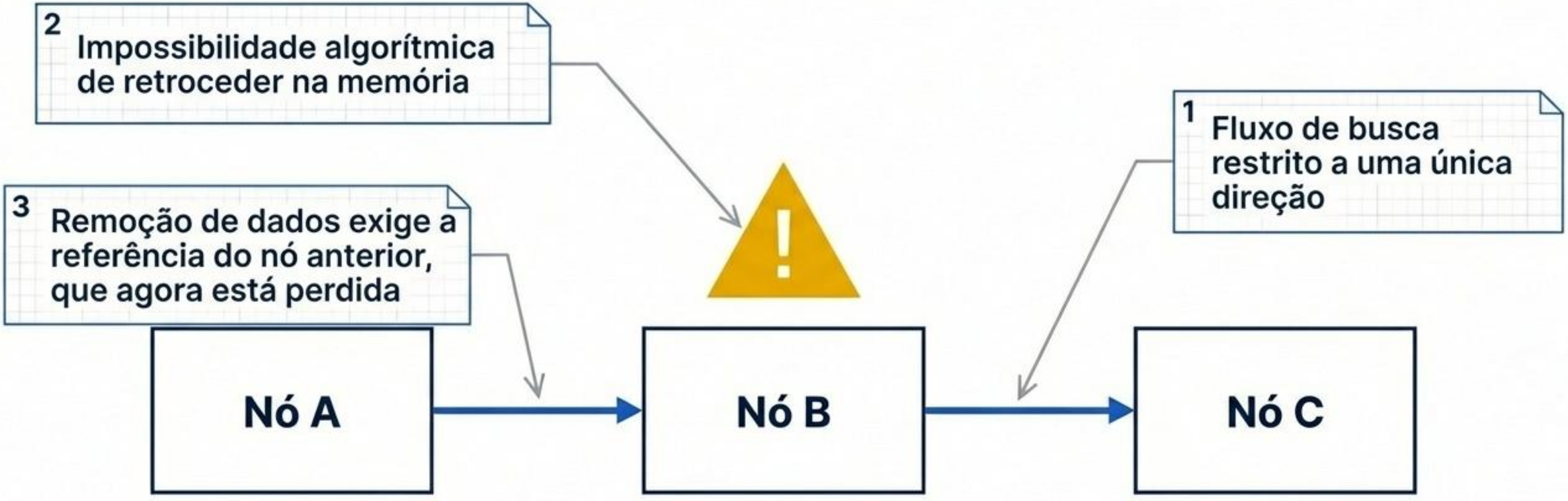
- Consolidação da prática de **Listas Simples**
- Validação de código e versionamento
- Introdução às estruturas bidirecionais

Validação Prática e Versionamento

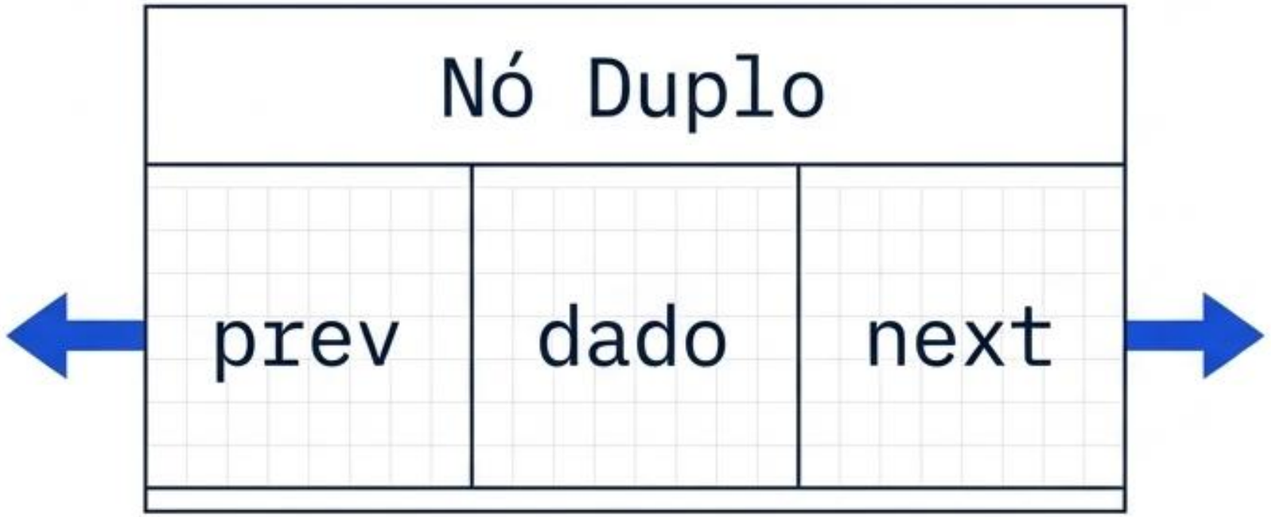


Regra de Ouro: Código que só existe na sua máquina local é código que não existe no mundo real.

O Limite da Navegação Unidirecional



Listas Duplamente Encadeadas



O(1)

Ganho

Adição do ponteiro bidirecional (prev) garante remoção e navegação com complexidade O(1).

Custos

- Maior consumo de memória RAM por nó individual.
- Maior complexidade na manutenção do código.

Minute Paper: Autoavaliação

- Responda individualmente em 3 minutos:

*Qual a principal diferença na **alocação de memória física** entre o **Vetor** e a Lista Encadeada?*

- Entregue o documento na mesa ao sair da sala.

Dúvidas?

CRIATIVIDADE
INOVAÇÃO
¡DEIA!
COMAMUNIDADE
CRIATIVIDADE
BRAINSTORM
INOVAÇÃO

ibmec

A direção do amanhã.

/ibmec